**Università degli Studi di Messina**
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

# HANDS-ON 13:
Condition Variables, Wait/Signal e Barriere

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

# Indice

**1 Problem Statement** **2**

**2 Stato dell'Arte** **2**

**3 Metodologia ed Implementazione** **2**
3.1 Simulazione di un Join (`ex1_join.c`) . . . . . 2
3.2 Barriera a Broadcast (`ex2_barrier.c`) . . . . . 3
3.3 Istruzioni di Esecuzione . . . . . 3

**4 Risultati** **4**
4.1 Output Simulazione Join . . . . . 4
4.2 Output Barriera a Variabile Condizionale . . . . . 4

**5 Conclusioni e Sviluppi Futuri** **4**

# 1 Problem Statement

L'Hands-On 13 chiude il cerchio e il quadro dei costrutti di sincronizzazione introducendo le **Condition Variables** e i meccanismi signal-driven. Lo scopo dell'esercitazione richiede la loro applicazione pratica lungo due scenari: 1. **Simulazione di una Thread Join**: Costruire una logica padre-figlio impiegando segnali di terminazione espliciti, interdicendo esplicitamente l'operazione `pthread_join()` e ricorrendo alle chiamate `wait()` e `signal()` per mettere in attesa il server master al compiersi dell'esatta direttiva asincrona dello slave. 2. **Barriera di Sincronizzazione con Broadcast**: Ricreare l'impianto della barriera (struttura vista all'Hands-on 12) rimuovendo il ricorso a cascatismi di *sem_post* su semafori ed integrando al suo posto la logica scalabile delle funzioni `broadcast()` delle variabili condizionali.

# 2 Stato dell'Arte

Le Variable Condition segnano un passaggio dai lock puramente "esclusivi" (Mutex) e limitativi (Semafori) ad una forma di cooperazione logica di alto livello.

# 3 Metodologia ed Implementazione

## 3.1 Simulazione di un Join (`ex1_join.c`)

Il meccanismo di attesa e segnalazione permette di coordinare l'esecuzione di due thread dipendenti senza occupare attivamente la CPU durante l'attesa (attesa passiva).

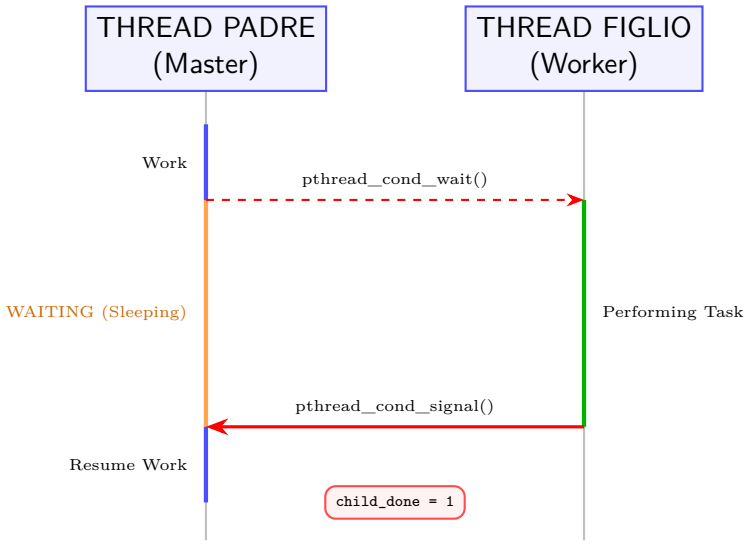


Figure 1: Diagramma di sequenza della sincronizzazione tramite Condition Variables (Wait/Signal).

Questa tipologia di codice mima al 100% la gestione in background dello scheduler per attendere flussi operativi prima di uscire in un `return 0;`.

**Snippet Esercizio 1: Padre vs Figlio**

```
// T-PADRE: Si blocca se non pronto
pthread_mutex_lock(&lock);
while (child_done == 0) {  // Uso un while ciclico di protezione base
    pthread_cond_wait(&cond, &lock);
}
// Se e' qui in basso, cond_signal e' arrivato e lock e' stato ripreso in mano!
pthread_mutex_unlock(&lock);

// T-FIGLIO: Avvisa la terminazione
pthread_mutex_lock(&lock);
child_done = 1;
pthread_cond_signal(&cond); // Rende udibile ai pendenti il cambiamento
pthread_mutex_unlock(&lock);
```

## 3.2 Barriera a Broadcast (`ex2_barrier.c`)

Prevede la sostituzione radicale nel *modus operandi* di blocco della barriera implementando la *funzione massiva POSIX d'ambiente*: cond_broadcast.

**Snippet Esercizio 2: Arrivo globale al punto zero**

```
if (count == N) {
    // Ultimo thread arrivato al checkpoint
    pthread_cond_broadcast(&cond); // -> Sveglia massivamente da 0 N thread in coda
} else {
    // Si accodano attendendo
    while (count < N) {
        pthread_cond_wait(&cond, &lock);
    }
}
```

## 3.3 Istruzioni di Esecuzione

L'applicativo viene gestito interamente tramite Makefile:

**Build & Run**

```
$ make
$ ./programma
```

# 4 Risultati

Le compilazioni e le catture da log sul terminale attestano il soddisfacimento dei vincoli imposti al design.

## 4.1 Output Simulazione Join

**System V Message Queues - Risultati**

```
massimo@PORTATILE-Linux:~/Scrivania/LabReSid24-25/hands-on/ho13$
   ./ex1_join
Padre: Creo il thread figlio.
Padre: In attesa della terminazione del figlio...
Thread Figlio: Iniziato. Faccio un po' di lavoro...
Thread Figlio: Lavoro terminato. Avviso il padre...
Padre: Il figlio ha confermato la terminazione (Join
   simulato con successo).
```

La dimostrazione concreta che il Thread chiamante non incorre ad uscita forzata nonostante la mancanza di una vera `pthread_join`. Questo garantisce lo shutdown dolce.

## 4.2 Output Barriera a Variabile Condizionale

**POSIX Message Queues - Risultati**

```
Thread 0: Arrivato alla barriera. (Presenti: 1/4)
Thread 1: Arrivato alla barriera. (Presenti: 2/4)
Thread 2: Arrivato alla barriera. (Presenti: 3/4)
Thread 3: Arrivato alla barriera. (Presenti: 4/4)
Thread 3: Sono l'ultimo arrivato! Sblocco tutti quanti!
Thread 3: Barriera superata! Avvio simultaneo.
Thread 1: Barriera superata! Avvio simultaneo.
Thread 0: Barriera superata! Avvio simultaneo.
Thread 2: Barriera superata! Avvio simultaneo.
```

Il trace del Thread 3 fa scattare atomicamente in concorrenza l'operazione successiva al blocco condizionale rendendo perfetta nel *timestamp* la coordinazione d'avvio.

# 5 Conclusioni e Sviluppi Futuri

Le Condition Variables sono apparse e si sono dimostrate infinitamente più pulite per le gestioni a coda massiva. Il Broadcast ne esalta a pieno la supremazia su semafori quando quest'ultimi sarebbero costretti a gestire la fine scalata (loop for di $N$ `sem_post`) o ad ingolfare l'handler chiamante limitando fortemente l'allargabilità orizzontale.
**Sviluppi futuri:**

- **Analisi Prestazionale**: Studio dellímpatto della latenza di sistema sulla scalabilità dellárchitettura.
- **Ottimizzazione delle Risorse**: Refactoring del codice per minimizzare lúso della CPU in contesti ad alta concorrenza.